NextWork.org

Build a Three-Tier Web App

Natasha Ong

Natasha Ong
NextWork Student

NextWork.org

Introducing Today's Project!

In this project, we will demonstrate how to set up a three-tier web app from scratch! We'll start with the presentation tier, then set up the logic tier and finally set up the data tier before tying them all together.

Tools and concepts

Services we used were Amazon S3, CloudFront, DynamoDB, Lambda, API Gateway. Key concepts we learnt include Lambda functions, CORS errors, updating the Javascript file with the API Invoke URL, and testing the invoke URL within the browser

Project reflection

This project took me approximately 2.5 hours including demo time. The most challenging part was resolving the CORS errors in both API Gateway and Lambda. It was most rewarding to see the final outcome - data getting returned in our web app!

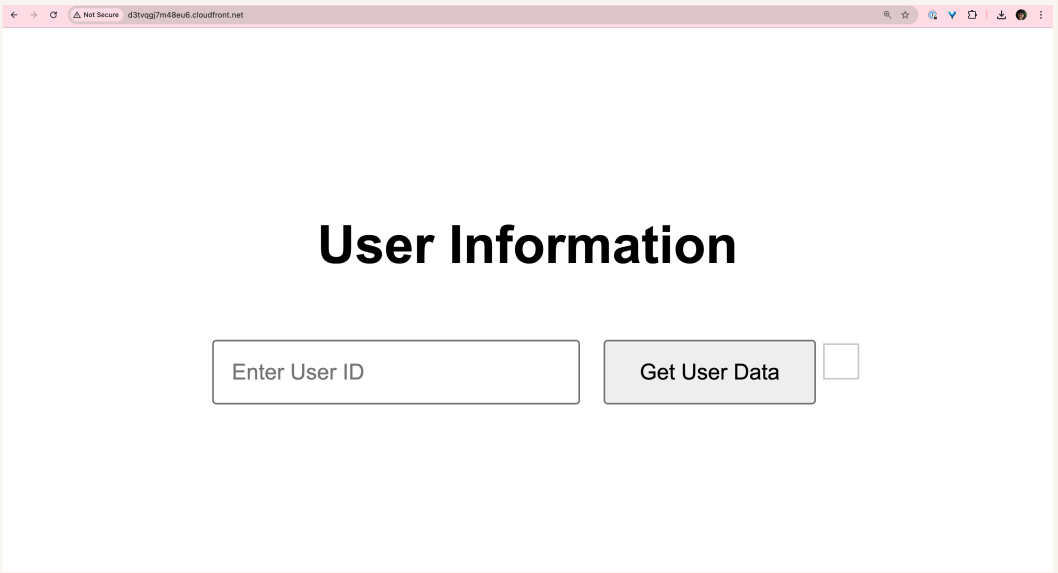
We dis this project today to learn about the three-tier architecture and set up our own web app. This project absolutely met our goals and we could see a fully functioning web app by the end.

Natasha Ong
NextWork Student

Presentation tier

For the presentation tier, we will set up how our website will be displayed and available to our end users. This is because the presentation tier is responsible for storing our website's files (Amazon S3) + website distribution (Amazon CloudFront).

We accessed our delivered website by visiting the CloudFront distribution's URL. This URL works because we also set up an origin access control that lets our S3 bucket restrict access to only our CloudFront distribution.

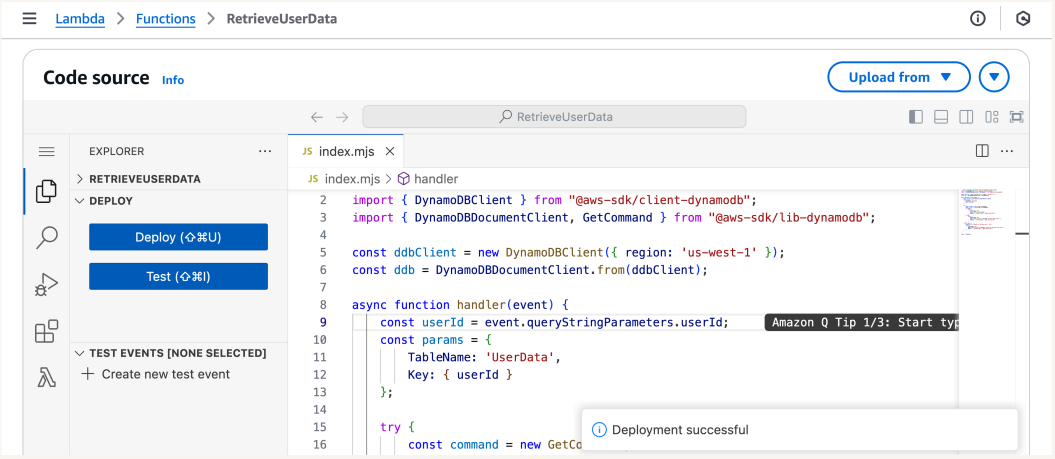


Natasha Ong
NextWork Student

Logic tier

For the logic tier, we will set up a Lambda function to handle requests (e.g. look up userId to return some user data) and also an API with API Gateway to receive requests from the user and hand it over to Lambda.

The Lambda function retrieves data by looking up a userId (that the user enters over the web app) in DynamoDB. The AWS SDK is used in the function code so we can use templates and libraries that lets us find the correct DynamoDB table + request data.



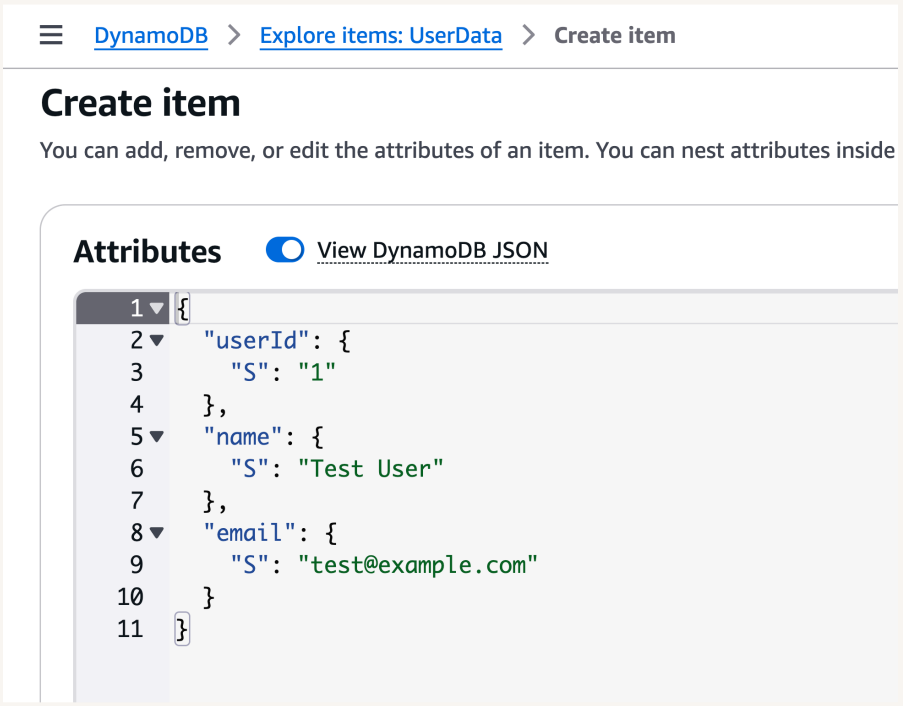
Natasha Ong
NextWork Student

NextWork.org

Data tier

For the data tier, we will set up a DynamoDB database that stores user data. At the moment, there is no user data for us to return to our web app's users. The data in our database will get returned once we set up DynamoDB and connect it with Lambda.

The partition key for our DynamoDB table is userId. This means that when our table looks up user data, it will look it up based on userId. Then, it can return all data (values) related to the item with that ID.



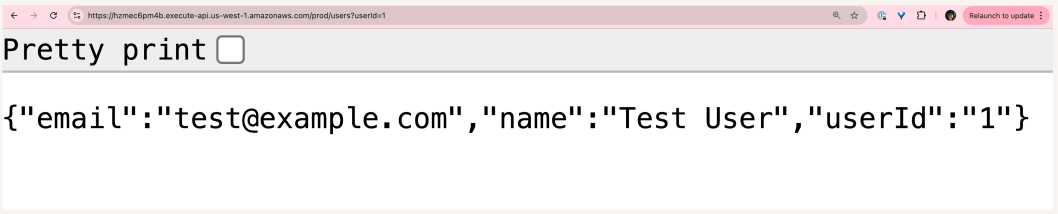
Natasha Ong
NextWork Student

NextWork.org

Logic and Data tier

Once all three layers of our three-tier architecture are set up, the next step is to connect the presentation and logic tier. This is because currently there is no way for our API to catch requests that users make through our distributed site!

To test our API, we visited the Invoke URL of the prod stage API. This let us test whether we can use the API and retrieve user data. The results were some user data in JSON when we looked up userId=1. This proved a logic+ data tier connection.



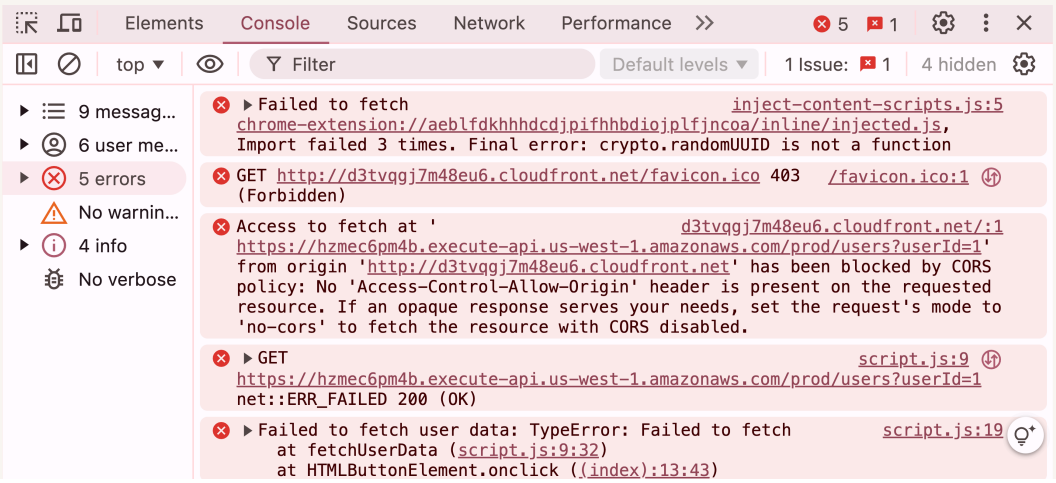
Natasha Ong
NextWork Student

Console Errors

The error in my distributed site was because there was an error within script.js (one of the website files I had uploaded into S3). The script.js file is referencing an prod stage API URL placeholder and not my API's actual URL.

To resolve the error, we updated script.js by replacing some placeholder text with the API's prod stage Invoke URL. We then reuploaded script.js into S3 because the S3 bucket is still storing the last uploaded version (i.e. with the error).

We ran into a second error after updating script.js. This was an error on the browser side with CORS because API Gateway, by default, is only configured to allow requests from users directly runing its Invoke URL in the browser (not with CloudFront).



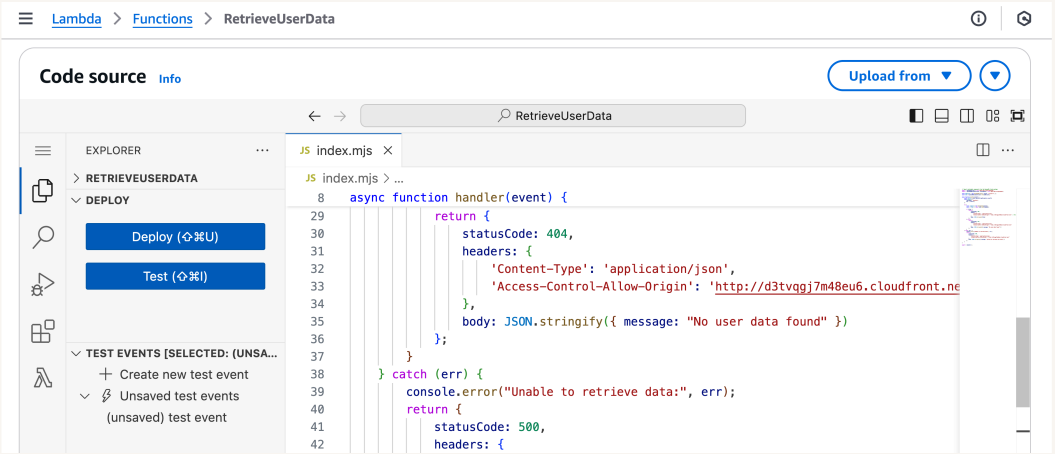
Natasha Ong
NextWork Student

NextWork.org

Resolving CORS Errors

To resolve the CORS error, we first went into our API (in API Gateway) and enabled CORS on the /users resource. We then made sure GET requests are enabled, and referenced our CloudFront domain as the domain getting access.

We also updated our Lambda function code because it needs to be able to return CORS headers to show that it has the permission to invoke the API's URL and return a response. We added 'Access-Control-Allow-Origin' as a header in the response.

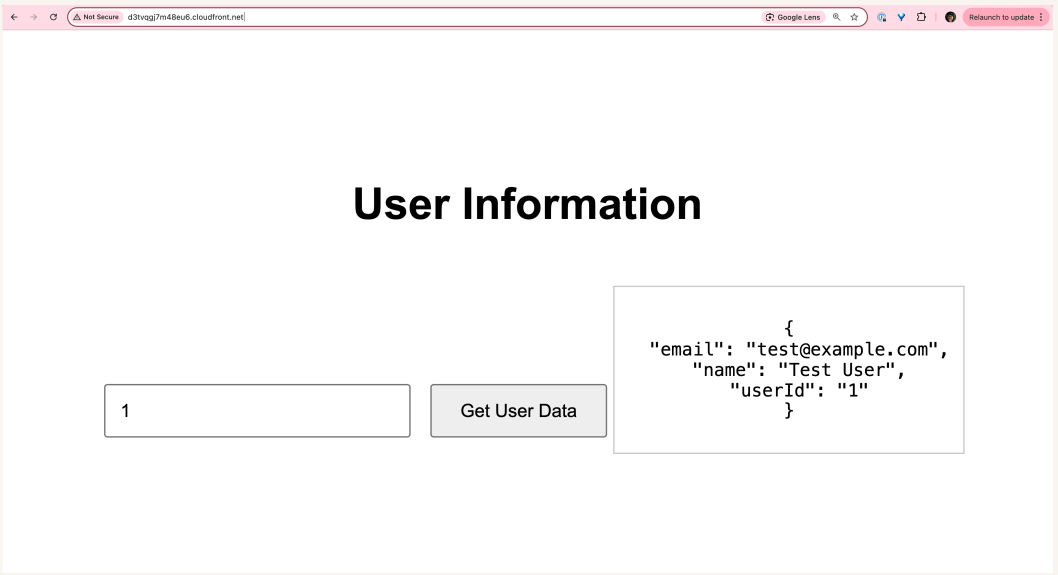


Natasha Ong
NextWork Student

NextWork.org

Fixed Solution

We verified the fixed connection between API Gateway and CloudFront by looking up user data in the distributed site again. In our final test, user data could be returned - so a user request in the presentation tier gets data from the data tier.



Everyone should be in a job they love.

Check out nextwork.org for more projects